HireX — Security Architecture & Access Control

Document ID: `HXP-HANDOVER-10`
Platform Version: HireX V0 / Enterprise Security & Compliance Baseline
Target Audience: Security Reviewers, Senior Developers, DevOps, Compliance Officers
Last Updated: October 2026

---

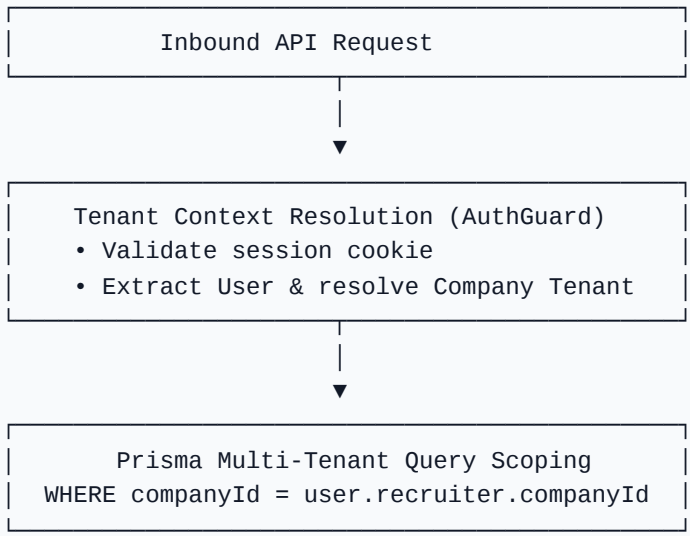
1. Security Architecture Principles

HireX is engineered around four core security pillars:

1. **Tenant Isolation by Default:** Every read/write query scoped to `companyId`.
2. **Fail-Closed Integration Security:** Missing credentials, invalid signatures, and unverified webhooks fail closed by throwing explicit errors rather than proceeding silently.
3. **Session Cookie Boundary (ADR-001):** Distinct role resolution preventing session hijacking or privilege escalation across the three distinct portals.
4. **Least-Privilege Role Authorization (RBAC):** NestJS guards enforcing strict method-level permissions.

---

2. Multi-Tenant Isolation Architecture



2.1 Enforced Query Boundaries

- Recruiter queries are permanently bound to `user.recruiter.companyId`.
- Even if an attacker supplies another company's UUID in route parameters (e.g. `GET /jobs/foreign-job-id`), the backend verifies ownership against the active session's `companyId` and throws `404`

`Not Found` or `403 Forbidden`.

- Automated test verification: Verified via `tenant-isolation.e2e-spec.ts`.

---

3. Session Security & ADR-001 Isolation

3.1 Cookie Partitioning & Flags

- **Cookie Name:** `better-auth.session_token`
- **Security Flags:**
  - `HttpOnly: true` (Inaccessible to browser JavaScript, mitigating XSS token theft)
  - `SameSite: Lax` (Mitigates Cross-Site Request Forgery / CSRF)
  - `Secure: true` in production environments (Requires HTTPS)
  - `Domain`: Scoped via `COOKIE_DOMAIN` (e.g. `.hire.zobay.in` in staging)

3.2 Mitigation of Privilege Escalation

In a recruitment suite with three separate frontends, a candidate signing in on `:3000` must never gain administrative access if they navigate to the recruiter URL `:3002`.

- When an incoming request reaches `/recruiters/*`, `AuthGuard` queries the authenticated user's assigned role.
- If `user.role !== 'RECRUITER'`, the request is immediately rejected with `403 Forbidden`.
- Automated test verification: Verified via `auth.guard.isolation.spec.ts`.

---

4. Webhook Cryptographic Verification (Fail-Closed)

4.1 Schedular (MeetX) Webhook Verification

All incoming webhooks at `POST /webhooks/schedular` must be verified using HMAC SHA-256 signatures:

```
const computedSignature = crypto
  .createHmac('sha256', process.env.SCHEDULAR_WEBHOOK_SECRET)
  .update(rawBody)
  .digest('hex');

if (computedSignature !== receivedSignature) {
  throw new UnauthorizedException('Invalid webhook signature');
}
```

- **Fail-Closed Protection:** If `SCHEDULAR_WEBHOOK_SECRET` is unset in production, the application refuses to boot or rejects all incoming webhooks outright.

- Automated test verification: Verified via `schedular-webhook.controller.spec.ts`.

---

5. Input Validation & Data Sanitization

1. **Global Validation Pipe:** Configured in `main.ts` with `whitelist: true`, `forbidNonWhitelisted: true`, and `transform: true`. Unrecognized fields in request payloads are automatically stripped.
2. **DTO Schemas:** Powered by `class-validator` (e.g. `@IsEmail()`, `@MinLength(8)`, `@IsUUID()`).
3. **File MIME-Type & Size Constraints:**
   - Resumes & Documents restricted to `application/pdf`, `image/jpeg`, `image/png`.
   - Maximum upload size strictly enforced at 10 MB.

---

6. Cloudflare R2 Storage Security

- **Private Buckets:** The Cloudflare R2 bucket (`hirex-test-dev`) is configured with private access. Direct public bucket listing is disabled.
- **Access Routing:** Resumes and sensitive compliance documents generate time-limited presigned URLs (`@aws-sdk/s3-request-presigner`) for authenticated viewers.
- **Cascade Integrity:** `Application.resume` utilizes `onDelete: SetNull` ensuring safe historical record preservation.

---

7. Security Gaps & Recommended Hardening

Component	Current State	Recommendation for V1
Public Guest Uploads	Protected by file extension and size filters	Add IP-based rate limiting via `@nestjs/throttler`
Login Attempts	Standard credential verification	Implement progressive backoff delays after 5 consecutive failed login attempts
CORS Origins	Configured to allowed portal URLs	Maintain strict production origin whitelist in cloud reverse proxy